



Error Handling

Error handling in Python is crucial for writing robust and reliable code. Python provides a mechanism for handling errors using try, except, else, and finally blocks.

```
try:
    # Code that may raise an exception
    # ...
except ExceptionType as e:
    # Handle the exception
    # ...
else:
    # Code to execute if no exception is raised
    # ...
finally:
    # Code to execute regardless of whether an exception was
    # ...
```

common exception classes that you might encounter and handle in your Python programs:

ValueError (Raised when an operation or function receives an argument of the correct type but an inappropriate value):

```
try:
    # code that might raise a ValueError
except ValueError as e:
    print(f"ValueError: {e}")
```

TypeError (Raised when an operation or function is applied to an object of an inappropriate type):

```
try:
    # code that might raise a TypeError
except TypeError as e:
    print(f"TypeError: {e}")
```

FileNotFoundError (Raised when a file or directory is requested but cannot be found):

```
try:
    # code that might raise a FileNotFoundError
except FileNotFoundError as e:
    print(f"FileNotFoundError: {e}")
```

ZeroDivisionError (Raised when the second operand of a division or module operation is zero):

```
try:
    # code that might raise a ZeroDivisionError
except ZeroDivisionError as e:
    print(f"ZeroDivisionError: {e}")
```

IndexError (Raised when a sequence subscript is out of range):

```
try:
    # code that might raise an IndexError
except IndexError as e:
    print(f"IndexError: {e}")
```

KeyError (Raised when a dictionary key is not found):

```
try:
    # code that might raise a KeyError
except KeyError as e:
    print(f"KeyError: {e}")
```

AttributeError (Raised when an attribute reference or assignment fails):

```
try:
    # code that might raise an AttributeError
except AttributeError as e:
    print(f"AttributeError: {e}")
```

OverflowError (Raised when an arithmetic operation exceeds the limits of the current Python interpreter):

```
try:
    # code that might raise an OverflowError
except OverflowError as e:
    print(f"OverflowError: {e}")
```

IOError (Base class for I/O-related errors):

```
try:
    # code that might raise an IOError
except IOError as e:
    print(f"IOError: {e}")
```

RuntimeError (Raised when an error is detected that doesn't fall in any of the other categories):

```
try:
    # code that might raise a RuntimeError
except RuntimeError as e:
    print(f"RuntimeError: {e}")
```

Exception (Base class for all exceptions):

```
try:
    # code that might raise any exception
except Exception as e:
    print(f"Exception: {e}")
```